

Preparcial: Sistema de Gestión de Alquiler de Vehículos

Contexto

Una empresa de alquiler de vehículos administra flotas entregadas en distintas sucursales. Cuando un cliente retira un vehículo, se registra un alquiler **ABIERTO**: se conoce cuántos días contrató, la tarifa diaria acordada y la franquicia de kilómetros incluida en esa tarifa.

El sistema procesa un archivo CSV con esos alquileres, valida su integridad, descarta los registros cancelados y calcula el importe de cada alquiler y los totales por sucursal.

Este proyecto se entrega aproximadamente un día antes del parcial para leerlo, ejecutarlo y comprenderlo. En esta instancia no hay que agregar funcionalidades: la evolución del modelo se pide recién durante el examen.

Estructura del proyecto

El proyecto Maven incluye las siguientes clases en el paquete `utnfc.backend.parcial`:

- **Alquiler**: representa un alquiler de vehículo, con sus atributos, invariantes y el cálculo del importe.
- **ParserAlquileres**: lee el archivo CSV, clasifica cada fila y construye los objetos válidos.
- **ResultadoParseo**: resume el resultado del procesamiento (alquileres leídos, procesados, descartados e inválidos).
- **Garage**: ofrece operaciones de consulta, filtrado y cálculo sobre la colección de alquileres.

Ejecución

Requiere JDK 21 o 25 y Maven 3.9.x. Desde la raíz de este proyecto:

```
mvn test
java -cp target/classes utnfc.backend.parcial.Main
java -cp target/classes utnfc.backend.parcial.Main datos/alquileres.csv
```

La primera ejecución puede requerir Internet para resolver dependencias. Luego puede probarse `mvn -o test`. El programa no necesita dependencias externas durante la ejecución.

Contrato del CSV

El archivo está codificado en UTF-8 y debe comenzar exactamente con:

```
id,cliente,categoria,sucursal,dias,tarifaDiaria,kmIncluidos,estado
```

Cada fila de datos tiene ocho campos separados por comas. No se contemplan comas, comillas ni saltos de línea dentro de un campo: es un CSV controlado para el ejercicio, no un lector de CSV universal. Se

quitan los espacios externos de cada campo y se conservan campos vacíos mediante `split(",", -1)`.

La empresa identifica cada alquiler con `id`, cliente, categoría de vehículo y sucursal. No se exige que los identificadores sean únicos: cada fila válida es un alquiler independiente. `dias` y `kmIncluidos` son enteros; `tarifaDiaria` es un número positivo.

Reglas del dominio

- `id`, `cliente`, `categoria` y `sucursal` no pueden estar vacíos.
- `dias` debe ser mayor que cero.
- `tarifaDiaria` debe ser mayor que cero.
- `kmIncluidos`, que representa la franquicia de kilómetros contratada, no puede ser negativo.
- El importe de un alquiler abierto es `dias * tarifaDiaria`: se cobra por los días contratados, porque todavía no se conoce el uso real del vehículo.

El objeto `Alquiler` protege estas invariantes. La conversión desde campos textuales y las validaciones propias del alquiler permanecen cerca del dominio, en `Alquiler.desdeCampos`, no en `Main`.

Estados y errores

- Una fila con estado `CANCELADO` se descarta después de verificar que la cantidad de columnas sea correcta, y no crea un objeto.
- Una fila con estado `ABIERTO` se intenta convertir en un `Alquiler`.
- Cualquier otro estado es inválido en este modelo inicial.
- Una fila con cantidad de columnas incorrecta es inválida.
- Un número no convertible, un campo obligatorio vacío o un valor fuera de rango vuelve inválida la fila.
- El parser registra la línea física y el motivo del error, pero continúa con las filas siguientes: una fila con problemas no aborta la carga completa.
- Un encabezado incorrecto aborta con `IllegalArgumentException`.
- Un error de lectura del archivo propaga `IOException`.

El encabezado no cuenta como fila leída. Para las filas de datos se cumple:

```
leídas = procesadas + descartadas + inválidas
objetos = procesadas
```

Se conserva el orden de los alquileres aceptados y de los diagnósticos.

Recorrido sugerido

1. Ejecutar `mvn test` y `Main`.
2. Leer `Alquiler` y observar sus validaciones y el cálculo de `importe()`.
3. Seguir `ParserAlquileres` y `ResultadoParseo` para entender la clasificación de filas.
4. Leer `Garage`, su copia defensiva, los filtros, el total y los totales por sucursal.
5. Ubicar en el CSV casos válidos, cancelados y filas con valores frontera.

El dataset tiene exactamente 60 filas de datos más el encabezado, es decir, 61 líneas. El programa debe poder mostrar las filas leídas, procesadas, descartadas e inválidas, el total general y los totales por sucursal.

Los tests usan JUnit 5. Los getters y la clase de resultado están escritos con Java convencional; no se agrega Lombok.

Nota: este material es preparatorio para el parcial. Durante el examen se solicitará una evolución del modelo que incorpora el cierre de un alquiler.